Just Noticeable Difference Modeling for Token Compression in Vision-Language-Action Models

Zhuoyuan Li, *Member, IEEE,* Rui Zhao, *Member, IEEE,* Jin Wang, Hanwei Zhu, *Member, IEEE,* Cong Zhang, *Member, IEEE,* Giuseppe Valenzise, *Senior Member, IEEE,* Weisi Lin, *Fellow, IEEE,* and Kin-Man Lam, *Senior Member, IEEE*

***Abstract*—Token compression has become a key technique for reducing the inference cost of large foundation models, with approaches such as token pruning and KV-cache reuse widely adopted in vision-language models and recently explored for embodied agents. In embodied agents, tokens are no longer used only for perception and semantic understanding, but also directly support latency-sensitive closed-loop robot action prediction. Existing schemes typically guide token compression using redundancy or importance cues, such as visual similarity, attention scores, and saliency. However, these cues only indirectly reflect the quantity that ultimately determines safe compression: how much a token can change before causing an unacceptable deviation in the downstream robot action. Such receiver-dependent tolerance is closely related to the core principle of just noticeable difference (JND) theory. Classical JND characterizes the tolerance to signal changes that remain imperceptible to the human visual system, while machine-oriented JND extends this tolerance boundary to downstream machine responses. Building on this progression, we introduce Action-JND, which extends JND modeling to embodied perception by defining noticeability through the language-conditioned action response of a vision-language-action (VLA) policy in closed-loop control, rather than through static downstream task responses in conventional machine perception. A visual-token change is therefore considered admissible only when the induced action deviation remains within a tolerated margin, rather than merely being imperceptible to humans or machines. To instantiate this concept, we develop a lightweight token-wise JND estimator in deep visual-feature space that predicts the maximum tolerable perturbation while preserving the policy's action response. The resulting action-tolerance score serves as a plug-and-play ranking signal for representative VLA compression paradigms, including stale-KV reuse and token pruning, prioritizing action-tolerant tokens for compression while preserving action-sensitive ones. Experiments on the LIBERO robotic manipulation benchmark with OpenVLA and OpenVLA-OFT demonstrate that Action-JND consistently improves compression reliability, with particularly substantial gains under aggressive compression ratios.**

***Index Terms*—Vision-language-action models, robotic manipulation, token compression, just noticeable difference**

Corresponding author: Rui Zhao.

Z. Li and K-M. Lam are with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong (e-mail: zhuoli@polyu.edu.hk, enkmlam@polyu.edu.hk)

R. Zhao, H. Zhu and W. Lin are with the School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798 (e-mail: zhao.rui@ntu.edu.sg, hanwei.zhu@ntu.edu.sg, wslin@ntu.edu.sg)

J. Wang is with the Bytedance Inc.

Cong Zhang is with the School of Control Science and Engineering, Shandong University (e-mail: congzhang@sdu.edu.cn)

Giuseppe Valenzise is with the CNRS, CentraleSupelec, Laboratoire des Signaux et Systèmes, Université Paris-Saclay, France (e-mail: giuseppe.valenzise@l2s.centralesupelec.fr).

I. Introduction

Large foundation models increasingly rely on long token sequences, making token compression a central challenge for efficient inference. In language models, long-context serving is often bottlenecked by the memory and bandwidth cost of the key-value (KV) cache, motivating KV-cache eviction, selection, and compression strategies that retain only a compact subset of informative historical tokens [1]–[4]. In multimodal large models, dense visual tokens further increase computational cost, and recent studies have explored visual token pruning, merging, and selection to reduce redundant computation while preserving downstream responses [5]–[10]. Meanwhile, feature coding provides an additional mechanism for compressing intermediate representations [11]–[17]. Similar efficiency challenges arise in vision-language-action (VLA) models for robotic manipulation, where robots must repeatedly process high-dimensional visual observations and generate actions in closed-loop control [18]–[21]. Reducing inference cost is particularly important in this setting because the policy operates within a closed-loop perception-action cycle, where lower latency enables more frequent action updates and more timely responses to environmental changes. Therefore, recent VLA acceleration methods have exploited spatial, semantic, and temporal redundancy through token pruning, layer pruning, dynamic scheduling, and adaptive cache reuse [22]–[29]. In this paper, following common usage in efficient foundation model inference, we use the term token compression broadly to refer to reducing the effective token-processing cost, rather than to conventional bitrate-oriented source coding.

Despite their methodological differences, most token compression methods follow a common principle: they estimate compressibility from measures of redundancy or importance. A token is more likely to be pruned or reused if it is visually similar to neighboring or previous tokens, receives low attention, or exhibits limited saliency according to a specific scoring rule. These signals are useful, but they provide only indirect evidence regarding whether compression is truly safe. Redundancy indicates the extent to which information overlaps with other tokens, while importance estimates the degree to which the model attends to a token. Neither directly addresses the question that ultimately matters: **if a token is pruned or replaced by a stale representation, will the downstream behavior remain within an acceptable margin?**

This question is closely related to a fundamental principle of compression: a representation can be altered as long as the resulting change remains acceptable to its intended re-

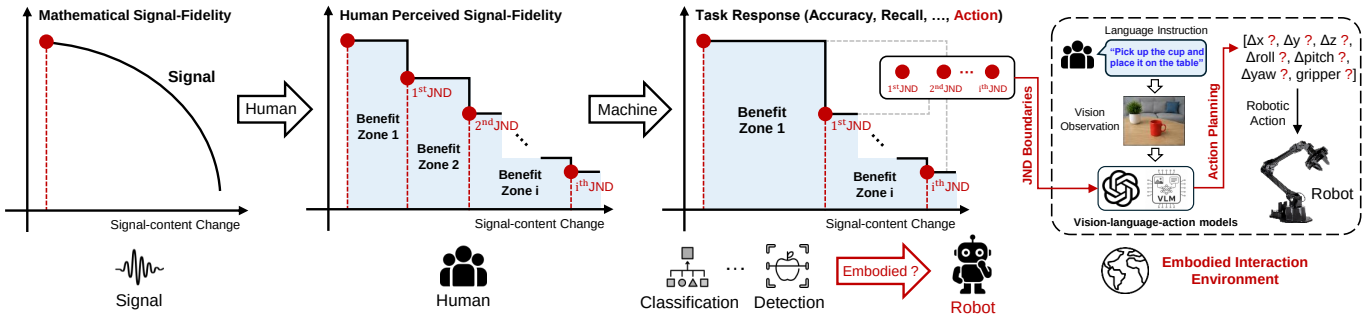


Fig. 1. **From classical signal-oriented JND to machine-oriented JND for embodied perception.** (1) Classical signal coding characterizes the trade-off between mathematical signal fidelity and signal-content change, where compression quality is usually evaluated using signal-level distortion. (2) Human-oriented JND introduces perceptual benefit zones, where signal changes remain unnoticeable to human observers despite measurable signal distortion. (3) Machine-oriented JND extends receiver-dependent tolerance to machine responses, where noticeability is determined by downstream task responses, such as classification and detection. (4) For embodied perception, we further instantiate the machine-oriented JND boundary with respect to action responses in a VLA-based embodied interaction environment, where noticeability is determined by whether changes in visual representations induce a tolerable deviation in the language-conditioned robot action during closed-loop control.

ceiver. In conventional image/video coding, this principle is usually expressed through the rate-distortion (R-D) trade-off, where lower bitrates are achieved at the expense of increased mathematical signal distortion [30]–[33], as shown in the leftmost part of Fig. 1. However, mathematical distortion alone does not necessarily indicate whether a compressed signal is perceptually degraded. Perceptual coding incorporates the human receiver into this trade-off, e.g., leveraging the just noticeable difference (JND) characteristics of the human visual system (HVS) [34]–[37], as shown in the second part of Fig. 1. Within the JND range, a signal may differ numerically from its original version yet remain visually indistinguishable to human observers, forming a benefit zone for compression. From this perspective, JND can be interpreted as a receiver-aware tolerance boundary that specifies the maximum admissible representation change without causing a perceptible variation.

This receiver-aware interpretation becomes more general when the compressed representation is consumed by a machine model rather than directly observed by humans. The compressed entity may take the form of a visual token, an intermediate feature representation, or a multimodal embedding, while the receiver is the downstream model that processes it. In such cases, the notion of "noticeability" should be defined by changes in the receiver's output response. As shown in the third part of Fig. 1, machine-oriented JND extends the benefit-zone concept to machine receivers, where the boundary is characterized by downstream task responses, such as classification accuracy, detection recall, or machine satisfaction ratios [38]–[42]. Recent studies have further expanded this idea to large multimodal models and deep visual features, where noticeability is quantified through multimodal response variation or task-preserving feature perturbations [43], [44]. These studies suggest that JND is not restricted to HVS but provides a general way for quantifying how much representation can change while preserving the response of a designated receiver.

Following this progression, embodied intelligence provides a new machine-response perspective for representation compression, as shown in the rightmost part of Fig. 1. In this setting, the machine receiver of compressed visual tokens is a language-conditioned robotic policy operating in a closed-loop control environment. Unlike conventional machine-perception tasks whose responses are typically class labels, bounding boxes, or textual outputs, the response of a robotic policy is a robot action, comprising end-effector translation $(\Delta x, \Delta y, \Delta z)$, rotation $(\Delta\text{roll}, \Delta\text{pitch}, \Delta\text{yaw})$, and gripper control (g). Therefore, the quality of token compression should be evaluated according to whether it induces action-level deviations, alters action planning, or degrades task execution, rather than according to image fidelity or generic task performance. Motivated by this observation, we formulate an action-conditioned tolerance boundary, termed **Action-JND**: the maximum compression-induced token change that preserves the robotic policy's action prediction within a tolerated deviation. Compared with existing machine-oriented JND formulations that mainly consider static downstream task responses [38]–[40], [44], Action-JND redefines the noticeability to robot actions in closed-loop control. Specifically, the receiver is instantiated as a VLA policy, the tolerated change is measured through action deviation, and the considered representation changes arise from token compression during VLA inference.

This action-level perspective provides a principled basis for re-examining existing VLA acceleration schemes in robotic control. During VLA inference, pruning a visual token or reusing a stale KV representation should be considered safe only when the resulting representation change remains within the Action-JND boundary. However, current token-level VLA acceleration methods usually approximate this safety criterion using indirect proxy signals. For example, VLA-Cache selects reusable tokens according to visual stability and attention-based risk estimates [22]; VLA-Pruner removes visual tokens based on temporal-aware semantic and action attention [28]; and EfficientVLA combines visual-token selection, layer pruning, and temporal feature caching to exploit redundancy throughout the VLA pipeline [23]. Recent methods have further explored model scheduling, self-speculative pruning, differentiable token pruning, and action-aware dynamic pruning [24]–[27]. These approaches effectively reduce computational cost by estimating token stability, importance, or redundancy, but they do not explicitly quantify the action tolerance of each token. This limitation becomes particularly pronounced under aggressive pruning or cache reuse. For instance, a visually stable token located near the gripper, contact region, or object boundary may be highly influential for action generation and therefore unsuitable for compression. Conversely, a semantically salient token may

admit substantial compression without affecting the predicted action. These observations suggest that token compression in VLA models should be guided not only by whether a token appears important or redundant, but also by how much its representation can change while preserving the policy's action response.

In this paper, we extend receiver-dependent JND modeling tailored to embodied perception through an action-conditioned tolerance boundary for VLA models. Unlike existing machine-oriented JND formulations that mainly consider static downstream task responses, Action-JND determines whether a token representation change is noticeable through the induced robot-action deviation in closed-loop control. To estimate this tolerance boundary, we instantiate Action-JND in the deep visual-feature space and develop a lightweight token-wise JND estimator on VLA models. During training, the JND estimator searches for token perturbations that are as large as possible while keeping the robotic policy's action prediction within an allowed deviation. The learned perturbation magnitude serves as an action-tolerance score, where high-score tokens are more action-tolerant and low-score tokens are more action-sensitive. Based on this score, we further develop two representative VLA acceleration paradigms: KV-cache reuse and token pruning. For KV-cache reuse, it ranks candidate tokens for stale-KV reuse according to their estimated action tolerance. For token pruning, Action-JND prioritizes the removal of highly action-tolerant tokens under a prescribed pruning budget. In both cases, the compression objective is shifted from preserving proxy cues to directly preserving downstream robot actions under compression. Extensive experiments across different compression ratios demonstrate that the proposed approach achieves superior performance over existing token-pruning and KV-cache-based VLA acceleration methods, with particularly substantial gains in aggressive compression regimes, resulting in more reliable acceleration and stronger action preservation.

The contributions of this paper are summarized as follows:

- We extend classical JND theory toward embodied perception, termed Action-JND, and formulate token compression in VLA models as an action-conditioned tolerance modeling problem, where noticeability is determined by the induced robot-action deviation.
- We instantiate the Action-JND in deep visual-feature space with a token-wise JND estimator, which learns tolerable token perturbations and converts their magnitudes into a criterion for token compression in VLA models.
- We integrate the learned Action-JND criterion into two representative VLA token compression paradigms, including KV cache reuse and token pruning, enabling more reliable stale-KV reuse and token removal.
- Extensive experiments across a broad range of compression ratios demonstrate that Action-JND improves compression reliability over existing caching and pruning schemes for VLA models.

II. Related Work

In this section, we review three research areas related to the proposed framework. First, we introduce recent VLA models and their action-generation paradigms. Then, we discuss efficient inference and token compression methods for large models, especially token pruning and temporal KV cache reuse in VLA models. Finally, we review JND modeling from human perception to machine-oriented tolerance estimation, which motivates our action-aware JND formulation for embodied perception.

A. Vision-Language-Action Models

VLA models extend vision-language foundation models from passive perception and language understanding to active robotic control. Instead of producing only textual responses, VLA models map visual observations and language instructions to executable robot actions. Early representative systems such as RT-2 formulate robot control as a vision-language generation problem by representing actions as language-like tokens and co-fine-tuning vision-language models with robotic trajectory data [18]. OpenVLA further advances this paradigm through an open-source 7B-parameter VLA model trained on large-scale robot demonstrations, where continuous robot actions are discretized into action tokens and generated autoregressively [19]. Other generalist robot policies and VLA-style models, such as Octo and π_0, explore different policy architectures and action representations to enable scalable robot control [21], [45]. Collectively, these studies show the potential of large pretrained multimodal models for language-conditioned robotic manipulation, while simultaneously exposing the substantial inference cost associated with large visual encoders and language backbones.

More recent efforts have focused on improving the efficiency and deployment practicality of VLA models by redesigning the action-generation process itself. OpenVLA-OFT studies several key fine-tuning choices, including autoregressive versus parallel decoding, discrete versus continuous action representations, and next-token prediction versus regression or diffusion-based objectives [20]. By combining parallel decoding, action chunking, continuous action prediction, and an ℓ_1 regression objective, OpenVLA-OFT improves both policy quality and action generation speed. These developments indicate that VLA models may adopt different action parameterizations, ranging from autoregressive discrete action tokens to continuous action chunks. Despite these advances, current VLA policies still rely on large visual encoders and language backbones and repeatedly process dense visual tokens during closed-loop control. Therefore, reducing inference cost while maintaining reliable action generation remains essential for practical VLA deployment, motivating the development of efficient inference and token-compression techniques.

B. Efficient Inference and Token Compression for Large Model

Token compression has been widely studied for efficient foundation-model inference. In large language models, KV-cache eviction, selection, and compression reduce the memory and bandwidth cost of long-context inference [1]–[4], while vision transformers and vision-language models reduce dense visual-token computation through token merging, pruning, and selection [5]–[10]. However, these methods are primarily

developed for generic language, vision, or multimodal inference, and their extension to embodied robotic control remains comparatively underexplored.

When token compression is applied to embodied agents, additional requirements emerge. VLA models and action policies operate within closed-loop robotic control systems and repeatedly process temporally adjacent observations to predict low-level actions under strict latency constraints. Since inference latency directly limits how frequently the policy can update robot actions, efficient inference is critical for maintaining responsive closed-loop control. Therefore, token compression must not only exploit spatial and temporal redundancy, but also preserve the visual information required for language-conditioned perception and action generation. Existing VLA acceleration methods can be broadly categorized into two groups. The first group improves model architectures or action-generation mechanisms, including parallel decoding, action chunking, quantization-aware pruning, and lightweight policy design [20], [29]. These approaches reduce computation by compacting the policy architecture, action-generation pipelines, or model parameters. However, dense visual-token representations are generally propagated through most of the VLA backbone, leaving a substantial amount of redundancy in intermediate visual representations unexploited. The second group performs token-level acceleration by exploiting redundancy within visual representations [22]–[28]. For example, VLA-Cache identifies visually stable tokens across consecutive observations and reuses their cached KV states, while using decoder attention to protect task-relevant regions [22]. VLA-Pruner jointly considers semantic understanding and action execution by combining vision-language prefill attention with temporally smoothed action-decoding attention for token pruning [28]. Other methods introduce model scheduling, self-speculative pruning, differentiable token pruning, action-aware dynamic pruning, and quantization-aware pruning to improve VLA efficiency [24]–[27], [29].

Although these approaches achieve promising acceleration performance, their compression decisions remain largely driven by proxy signals, such as visual stability, attention-based importance, scheduling confidence, or token redundancy. While such cues are useful for identifying potentially compressible tokens, they do not explicitly estimate the action sensitivity of individual tokens, namely the extent to which a token can be altered before the predicted robot action is affected. This limitation is particularly important in robotic manipulation scenarios, where visually stable or weakly attended regions may nevertheless play a critical role in action generation. Therefore, VLA token compression requires a criterion that directly evaluates whether a token can be pruned or reused while maintaining the resulting action deviation within an acceptable range. This motivates the proposed Action-JND formulation, which reframes token compression as an action-tolerance modeling problem.

C. JND Modeling

JND was originally introduced in human perception to characterize the minimum signal change that becomes perceptible to human observers. In image and video processing, JND has been widely used in perceptual coding, where distortions below the human JND threshold are mathematically measurable but visually unnoticeable, forming a benefit zone for compression, watermarking, quality assessment, and resource allocation [34]–[36]. Classical visual JND models mainly exploit characteristics of HVS, including luminance adaptation and contrast sensitivity. Subsequent studies further construct subjective JND datasets and develop learning-based predictors for picture-wise JND assessment, JND-critical region localization, and JND map estimation [46]–[49]. These works establish a fundamental principle of JND modeling: safe distortion should be determined by the receiver's response, rather than by signal fidelity alone.

This receiver-dependent perspective has gradually been extended from human perception to machine perception. In deep machine vision, DMV-JND shows that images exhibiting large signal distortion can still be tolerable for classification models [38]. For machine-oriented visual coding, the concepts of just recognizable distortion and satisfied machine ratio further characterize whether compressed visual signals preserve downstream machine-task performance [39], [40]. More recently, JND-based tolerance modeling has been extended to large multimodal models and deep visual features. LMM-JND investigates the minimum visual change that can be perceived by large multimodal models and reflected in their textual responses [43], whereas FeatJND estimates the maximum tolerable feature perturbation that can be introduced while preserving downstream classification, detection, and instance segmentation performance [44]. These studies show that JND is no longer limited to human perception, but can serve as a general tolerance modeling principle for machine receivers.

However, existing machine-oriented JND formulations mainly define noticeability through recognition outputs, multimodal responses, or static downstream task performance, while JND modeling for embodied perception remains largely unexplored. In embodied perception, the machine receiver interacts with the environment and produces robot actions conditioned on both visual observations and language instructions. Therefore, the tolerance of a visual representation change should be determined by its induced action deviation rather than by signal distortion or generic task accuracy. Motivated by this observation, we introduce Action-JND, which further extends receiver-dependent JND modeling to embodied perception through action-conditioned noticeability. This perspective further provides a receiver-aware criterion for reliable token compression in robotic manipulation.

III. JND Modeling for Embodied Perception

A. Preliminaries

JND characterizes the minimum signal change that becomes noticeable to a receiver. Equivalently, it can be interpreted as the maximum distortion that can be introduced while remaining unnoticeable under a given response criterion. In classical visual perception, the receiver is the HVS, and a signal change is considered imperceptible if it does not cause noticeable perceptual degradation [34]–[36]. Therefore, JND defines a receiver-dependent tolerance boundary: a distortion

is regarded as acceptable not because it is numerically small, but because it does not alter the receiver's perception. This receiver-dependent principle has subsequently been extended from human perception to machine perception, where noticeability is determined by changes in downstream machine responses. Existing machine-oriented JND studies examine whether perturbations applied to visual signals preserve recognition performance or machine satisfaction ratios [38]–[40]. More recently, FeatJND further extends this principle from input signals to intermediate feature representations [44]. Given an intermediate feature tensor f and a downstream task head $h(\cdot)$, FeatJND estimates a perturbation map δ that is as large as possible while preserving the downstream task response. This formulation shifts the tolerance criterion from input-signal changes to task-preserving feature perturbations, providing a basis for modeling JND over token representations.

Building upon this progression, we further extend JND modeling toward embodied perception. In VLA-based robotic manipulation, the machine receiver is instantiated as a robotic policy that consumes visual representations together with language instructions to generate control actions [18]–[21]. The receiver response to be preserved is therefore the language-conditioned robot action produced by the policy. Accordingly, a change in a visual-token representation becomes noticeable if the predicted action deviates beyond an acceptable tolerance level. We therefore define Action-JND as the maximum visual-token perturbation that can be introduced while preserving the action response of a VLA policy.

B. Receiver-Dependent Formulation

To formulate the above intuition, we first express JND in a general receiver-dependent form. Let x denote an input representation and Δ denote a signal or feature perturbation. For a receiver $\mathcal{R}$, let $r_{\mathcal{R}}(x)$ denote its response to x, and let $D_{\mathcal{R}}(\cdot,\cdot)$ measure the discrepancy between two responses. The specific form of $D_{\mathcal{R}}$ depends on the receiver and its response space. For example, it may represent perceptual difference for human observers, task discrepancy for machine perception models, or an action discrepancy for robotic policies. A receiver-dependent JND can be formulated as,

$$\Delta^{\star}_{\mathcal{R}} = \arg\max_{\Delta} \; M(\Delta), \quad \text{s.t.} \;\; D_{\mathcal{R}}\big(r_{\mathcal{R}}(x), r_{\mathcal{R}}(x+\Delta)\big) \le \epsilon_{\mathcal{R}}. \tag{1}$$

where $M(\Delta)$ measures the perturbation magnitude, such as ℓ_1 or ℓ_2 norm. $\epsilon_{\mathcal{R}}$ is the tolerated response variation. Here, $M(\cdot)$ and $D_{\mathcal{R}}(\cdot,\cdot)$ are scalar-valued functions, $\epsilon_{\mathcal{R}}$ is a scalar tolerance threshold, and x and Δ are vectors or tensors. For human-oriented JND, $\mathcal{R}$ is the HVS, and the constraint requires perceptual indistinguishability.

For machine-oriented JND, the receiver becomes a downstream machine task [44]. Let f denote an intermediate feature tensor and $h(\cdot)$ denote the downstream task head. The feature-level JND perturbation can be formulated as,

$$\delta^{\star} = \arg\max_{\delta} \; M(\delta), \quad \text{s.t.} \;\; D_{\text{task}}\big(h(f), h(f+\delta)\big) \le \epsilon_{\text{task}}, \tag{2}$$

where $D_{\text{task}}(\cdot,\cdot)$ measures the downstream task discrepancy, and ϵ_{task} denotes the tolerated task variation. Depending on application, D_{task} can be instantiated by different forms, such as classification loss, regression error, distribution divergence, or other task-specific measures. Eq. (2) instantiates the general receiver-dependent formulation in Eq. (1) for machine perception, where the receiver is a downstream task head and the response criterion is defined by task-response preservation.

C. Action-JND Formulation

For embodied perception, we instantiate the machine receiver as a VLA policy operating in a robotic system. At timestep t, let $F_t = [f_{t,1}, f_{t,2}, \ldots, f_{t,N}]^{\top} \in \mathbb{R}^{N\times D}$ denote the visual-token features, where N is the number of visual tokens and D is the feature dimension. Let ℓ denote the language instruction. A frozen VLA policy π_θ predicts the robot action as $a_t = \pi_\theta(F_t, \ell)$. For the robotic manipulation setting considered in this work, continuous-action policies can represent the action by end-effector translation, rotation, and gripper control [19], [20]: $a_t = [\Delta x_t, \Delta y_t, \Delta z_t, \Delta\text{roll}_t, \Delta\text{pitch}_t, \Delta\text{yaw}_t, g_t]$. Here, a_t is a vector whose components are scalars. For discrete-action generation policies, the response can instead be represented by action-token distributions [19]. The Action-JND formulation is not restricted to this action space and can be extended to other embodied agents by defining the policy response and action discrepancy according to their physical action spaces. For simplicity, we use a single-step action in the formulation, while the same response-level formulation applies to multi-step action chunks, as instantiated by OpenVLA and OpenVLA-OFT, respectively. Let $\tilde{a}_t = \pi_\theta(F_t + \Delta_t, \ell)$ denote the action predicted from perturbed features. We define action-level noticeability according to the discrepancy between the clean and perturbed policy responses, $D_{\text{act}}\big(\mathcal{R}_\theta(F_t,\ell), \mathcal{R}_\theta(F_t+\Delta_t,\ell)\big)$. Following the same JND principle [44], Action-JND is defined as the maximum visual-token perturbation that remains unnoticeable to VLA policy:

$$\begin{aligned} &\Delta^{\star}_t = \arg\max_{\Delta_t} \; M(\Delta_t), \\ &\text{s.t.} \quad D_{\text{act}}\big(\mathcal{R}_\theta(F_t,\ell), \mathcal{R}_\theta(F_t+\Delta_t,\ell)\big) \le \epsilon_a, \end{aligned} \tag{3}$$

where $\mathcal{R}_\theta(\cdot)$ denotes the policy response to be preserved. For discrete-action policies [19], it is the action-token distribution $P_\theta(a \mid F_t, \ell)$; for continuous-action policies [20], it is the regressed action vector $a_t = \pi_\theta(F_t, \ell)$. ϵ_a denotes the tolerated action deviation. Compared with feature-level JND in Eq. (2), Action-JND follows the same receiver-dependent JND principle but shifts the response criterion toward embodied action generation: the machine receiver is instantiated as a VLA policy rather than a task head, and the response is a robot action in closed-loop control rather than a classification, detection, segmentation, or textual output.

The action discrepancy D_{act} is instantiated according to how VLA policy parameterizes its action output. For discrete-action policies such as OpenVLA [19], the response is represented by the action-token distribution $P_\theta(a \mid F_t, \ell)$, and D_{act} is its Kullback-Leibler (KL) divergence under perturbation:

$$D_{\text{act}} = D_{\text{KL}}\Big(P_\theta(a|F_t,\ell) \;\big\|\; P_\theta(a|F_t+\Delta_t,\ell)\Big). \tag{4}$$

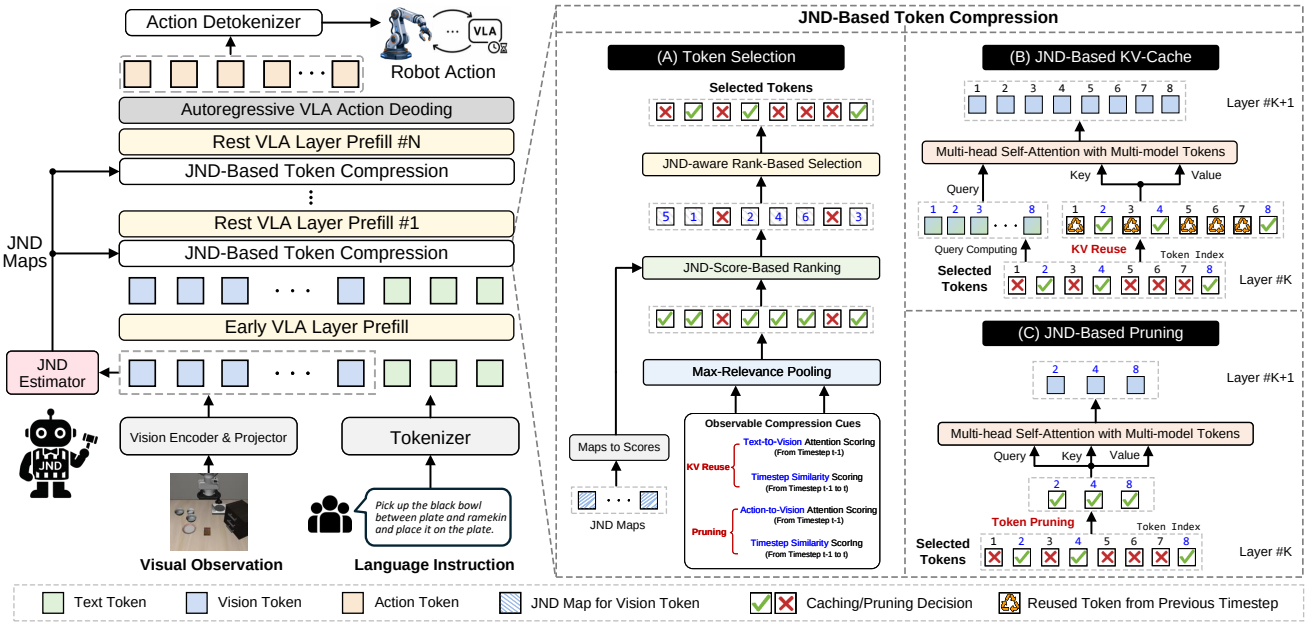


Fig. 2. **Overall framework of Action-JND-aware token compression.** Given a visual observation and a language instruction, the VLA model extracts visual tokens and language tokens for action prediction. A trainable Action-JND estimator predicts token-wise perturbation maps from the projected visual-token features. These maps are converted into action-tolerance scores that quantify the action tolerance of individual tokens. (1) The token-selection module ranks candidate visual tokens according to their JND scores, optionally together with framework-specific compression cues. (2) For KV-cache reuse, action-tolerant tokens reuse their cached key/value states from previous control steps, while action-sensitive tokens are recomputed. (3) For token pruning, highly action-tolerant tokens are removed from subsequent computation, whereas action-sensitive tokens are preserved.

For continuous-action policies that directly regress action chunks, such as OpenVLA-OFT [20], the action instead lies in a continuous space, and D_{act} is computed as a weighted ℓ_1 regression distance:

$$D_{\text{act}}(a_t, \tilde{a}_t) = \lambda_p \|\Delta p_t - \Delta \tilde{p}_t\|_1 + \lambda_r \|\Delta r_t - \Delta \tilde{r}_t\|_1 + \lambda_g |g_t - \tilde{g}_t|. \tag{5}$$

where Δp_t denotes the end-effector translation, Δr_t denotes the end-effector rotation, and g_t denotes the gripper command. λ_p, λ_r, and λ_g weight the translation, rotation, and gripper-control discrepancies, respectively.

Following the optimization strategy adopted in feature-level JND [44], Eq. (3) can be relaxed into an unconstrained objective that balances perturbation magnitude and action preservation:

$$\max_{\Delta_t} \ \lambda_{\text{mag}} M(\Delta_t) - \lambda_{\text{act}} D_{\text{act}}\big(\mathcal{R}_\theta(F_t, \ell), \mathcal{R}_\theta(F_t + \Delta_t, \ell)\big), \tag{6}$$

where λ_{mag} and λ_{act} control the trade-off between maximizing the allowed perturbation and preserving the predicted robot action. Eq. (6) provides a formulation for learning Action-JND. The objective jointly encourages large token-wise perturbations while constraining their overall effect on the policy response. The resulting perturbation magnitude therefore characterizes the token's tolerance to action-preserving perturbations and forms the basis of the learnable Action-JND estimator introduced in the next section.

IV. Action-JND-Aware Token Compression

A. Overview

After formulating Action-JND as an action-conditioned tolerance boundary, we further instantiate it as a learnable token-wise criterion for efficient VLA inference. As illustrated in Fig. 2, the proposed framework consists of three stages. First, given the projected visual-token features of a frozen VLA policy, an Action-JND estimator is trained to predict token-wise feature perturbations that are as large as possible while preserving the policy's action response. Second, during inference, the perturbation map generated by learned JND estimator is converted into token-wise action-tolerance scores. A larger score indicates that the corresponding token can tolerate a larger representation change without noticeably affecting the predicted action. Third, the learned score is used as an action-aware ranking criterion to guide token compression, including both temporal KV-cache reuse and direct visual token pruning.

The key idea is that the proposed Action-JND estimator serves as a plug-and-play module that can be integrated into existing token compression frameworks. It does not replace the original observable compression cues, such as temporal similarity, attention-based relevance, or token redundancy. Instead, it provides an additional criterion that directly reflects the impact of a token on the downstream robot action. Specifically, existing compression cues are first used to identify candidate tokens, and Action-JND then ranks these candidates according to their estimated action tolerance for subsequent KV-cache reuse or token pruning.

B. Learning Token-Wise Action-JND Maps

This subsection details how token-wise Action-JND maps are learned for VLAs. The architecture of the Action-JND estimator is first introduced, followed by the learning objective used to optimize the estimator under a frozen VLA policy.

1) Action-JND Estimator Architecture: Action-JND is instantiated with a lightweight estimator that predicts token-wise perturbations in the projected visual-feature space. Given visual-token features $F_t = [f_{t,1}, f_{t,2}, \ldots, f_{t,N}]^\top \in \mathbb{R}^{N \times D}$ extracted by the frozen VLA model, the estimator G_ϕ predicts $\hat{\Delta}_t = G_\phi(F_t)$, a learned approximation to the optimal

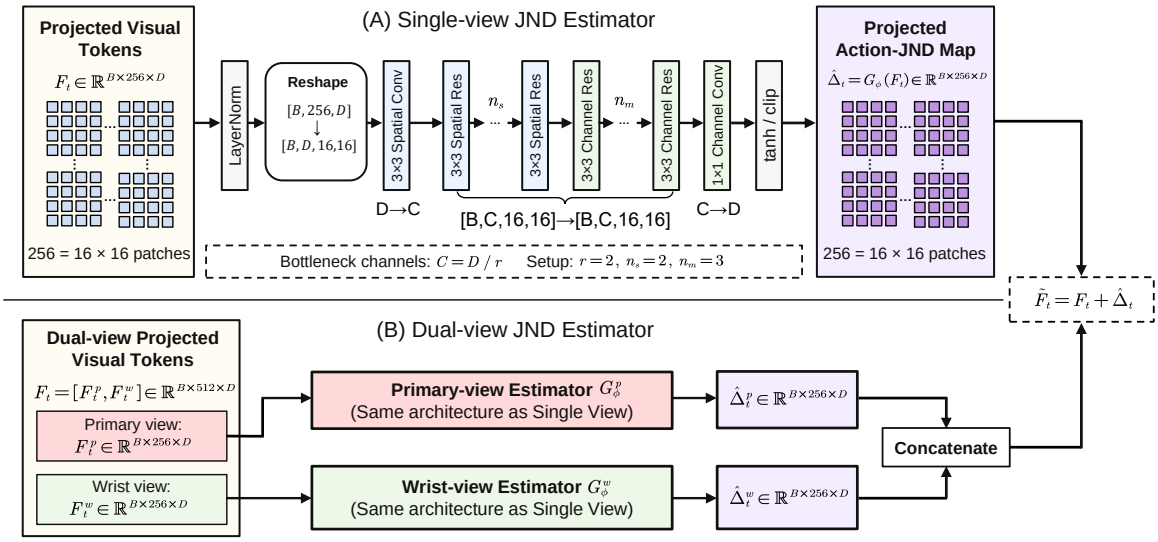


Fig. 3. **Architecture of the Action-JND estimator.** For the single-view setting, projected visual tokens are reshaped into a 16×16 spatial feature map and processed by a lightweight convolutional architecture consisting of a 3×3 spatial convolution, multiple spatial residual blocks, channel-mixing residual blocks, and a 1×1 output projection. The predicted perturbations are then reshaped back into token space to form a token-wise Action-JND map. For the dual-view setting, primary-view and wrist-view tokens are processed by two independent JND estimators with the same architecture but independent parameters, and their outputs are concatenated to produce the final Action-JND map.

perturbation $\Delta_t^\star$ in Eq. (3), with $\tilde{F}_t = F_t + \hat{\Delta}_t$. Here, F_t and $\tilde{F}_t$ denote the clean and perturbed visual-token representations, respectively. The estimator is designed to produce token-wise perturbations that are large in magnitude while remaining action-tolerable to the frozen VLA policy.

The architecture of G_ϕ is illustrated in Fig. 3. For a single-view VLA model, the visual tokens form a 16×16 spatial grid with $N = 256$. To preserve this spatial structure, $LayerNorm$ is first applied to the projected visual tokens, and the token sequence is reshaped from $[B, 256, D]$ to $[B, D, 16, 16]$. A spatial convolution then projects the channel dimension from D to C, where $C = D/r$ denotes the bottleneck dimension. The resulting feature map is processed by n_s spatial residual blocks for local relationship modeling, followed by n_m channel-mixing residual blocks for cross-channel interaction. Finally, a 1×1 projection maps the feature dimension back from C to D, and the output is reshaped back to token space and bounded by a $tanh/clipping$ function.

For dual-view VLA models, the projected visual features consist of primary-view tokens and wrist-view tokens: $F_t = [F_t^P, F_t^W] \in \mathbb{R}^{B\times512\times D}$, where $F_t^P, F_t^W \in \mathbb{R}^{B\times256\times D}$ each correspond to a 16×16 spatial grid. Two JND estimators with the same architecture but independent parameters, denoted by G_ϕ^P and G_ϕ^W, are used to process the two views separately:

$$\hat{\Delta}_t^P = G_\phi^P(F_t^P), \quad \hat{\Delta}_t^W = G_\phi^W(F_t^W). \tag{7}$$

The final perturbation map is obtained by concatenating the two view-specific outputs: $\hat{\Delta}_t = [\hat{\Delta}_t^P, \hat{\Delta}_t^W] \in \mathbb{R}^{B\times512\times D}$. Depending on the training strategy, the estimator can also be applied only to the primary view while keeping the wrist-view tokens unperturbed. This view-specific design preserves the spatial topology of each camera view and avoids mixing different visual coordinate systems.

2) Learning Objective: Fig. 4 illustrates the training procedure of the Action-JND estimator. During training, the VLA policy π_θ is kept frozen, while only the estimator G_ϕ is updated. The clean and perturbed visual features are fed into the same frozen policy to obtain two action responses: $r_t = \mathcal{R}_\theta(F_t, \ell)$, $\tilde{r}_t = \mathcal{R}_\theta(\tilde{F}_t, \ell)$, where ℓ denotes the language instruction. Following the unconstrained Action-JND objective in Eq. (6), the estimator is trained to encourage large perturbations while penalizing the resulting action discrepancy:

$$\mathcal{L}_{\text{JND}} = \lambda_{\text{act}} \cdot D_{\text{act}}\big(r_t, \tilde{r}_t\big) - \lambda_{\text{mag}} \cdot M(\hat{\Delta}_t), \tag{8}$$

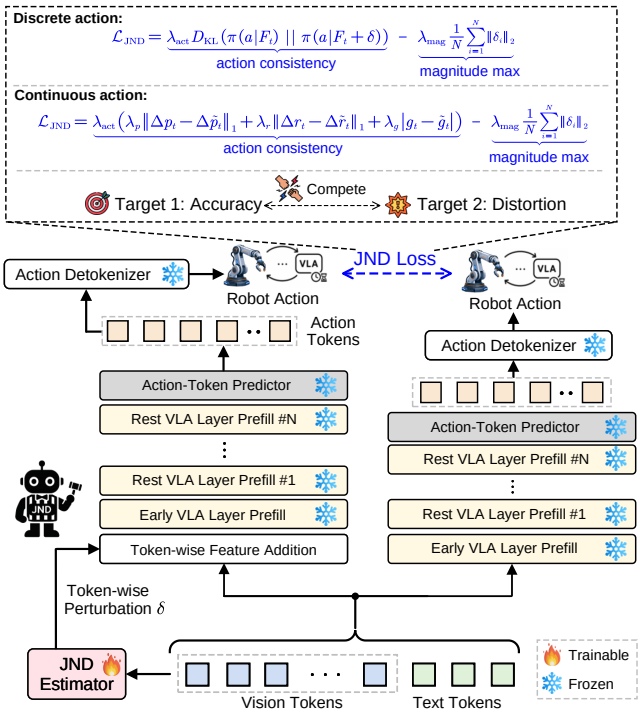


Fig. 4. **Training pipeline of the Action-JND estimator.** The VLA policy is frozen, and only the JND estimator is trainable. Given visual tokens and language tokens, the clean branch produces the original robot action, while the perturbed branch injects the predicted Action-JND perturbation into the visual-token features and produces a perturbed action response. The Action-JND loss encourages action consistency between the clean and perturbed branches while maximizing perturbation magnitude. The figure illustrates the KL-based objective for discrete action-token prediction; for continuous-action policies, the action discrepancy is instantiated using an action-regression distance.

where λ_{act} and λ_{mag} balance action consistency and perturbation magnitude. The perturbation-magnitude term is computed as $M(\hat{\Delta}_t) = \frac{1}{N}\sum_{i=1}^{N} \|\hat{\delta}_{t,i}\|_2$, where $\hat{\delta}_{t,i}$ denotes predicted perturbation for token i. This term instantiates the perturbation measure $M(\cdot)$ introduced in Eq. (1).

The action discrepancy D_{act} is instantiated according to the action parameterization of the VLA policy. For OpenVLA-style discrete action-token prediction [19], D_{act} is computed as the KL divergence between clean and perturbed action-token distributions. For OpenVLA-OFT-style continuous action regression [20], D_{act} is computed using an action regression distance, such as the weighted ℓ_1 distance defined in Eq. (5). Thus, the same training framework can be applied to both discrete action-token prediction and continuous action regression, with only the action-discrepancy term changed.

C. From JND Maps to Token Scores

After training, the predicted perturbation is converted into a token-wise JND score. Two score forms can be derived:

$$s_{t,i}^{\text{abs}} = \|\hat{\delta}_{t,i}\|_2, \qquad s_{t,i}^{\text{rel}} = \frac{\|\hat{\delta}_{t,i}\|_2}{\|f_{t,i}\|_2 + \varepsilon}, \tag{9}$$

where ε is a small constant introduced for numerical stability. Both scores share the same interpretation: a larger value indicates that the corresponding token can tolerate a larger feature perturbation while preserving the policy's action response. In our implementation, the relative score $s_{t,i}^{\text{rel}}$ is used for KV-cache reuse to reduce the influence of token-feature scale, whereas the absolute score $s_{t,i}^{\text{abs}}$ is used for direct visual token pruning. During inference, the predicted perturbation $\hat{\Delta}_t$ is not injected into the visual features. Instead, it is used only to estimate token-wise Action-JND scores. The clean projected features F_t are passed to the VLA policy for action prediction, ensuring that Action-JND affects inference only through token selection decisions.

Let $\mathcal{S}_t = \{s_{t,i}\}_{i=1}^{N}$ denote the token-wise Action-JND score map. This score serves as a ranking signal over the candidate set provided by a generic compression framework. Given an admissible candidate set $\mathcal{C}_t$, candidate tokens are ranked in descending order of their action tolerance:

$$\text{Rank}_{\text{JND}}(i) = \text{rank}_{\downarrow}\left(s_{t,i}\right), \qquad i \in \mathcal{C}_t. \tag{10}$$

Therefore, Action-JND refines the ordering among tokens that are already considered eligible for compression. When the candidate pool is insufficient, the same JND ranking can also be applied to framework-specific fallback candidates.

D. JND-Aware Token Compression

Here we present how the Action-JND scores are incorporated into token compression pipelines. The proposed criterion is not intended to redesign the compression framework. Instead, it introduces an action-aware signal into the token-selection procedure of existing methods. For a ranking-based token compression pipeline, let $\mathcal{C}_t$ denote the candidate token set generated by a generic compression framework, where the candidate tokens may be identified based on temporal stability, attention-based relevance, or other framework-specific cues. Given a target compression budget κ_t, the number of tokens that can be directly selected from the candidate set is $\bar{\kappa}_t = \min(\kappa_t, |\mathcal{C}_t|)$. The Action-JND-selected token set is then computed as $\mathcal{Q}_t = \text{TopK}_{i\in\mathcal{C}_t}\left(s_{t,i}, \bar{\kappa}_t\right)$, where tokens with larger Action-JND scores are prioritized for compression because they exhibit stronger action-level tolerance.

This formulation provides a unified plug-and-play interface for different VLA token compression mechanisms. In this work, it is instantiated in two representative compression scenarios: KV-cache reuse and token pruning. Although these two operations affect the VLA inference process in different ways, they share the same underlying principle: tokens with larger JND scores can be compressed because their representations are less critical to action generation.

JND-Aware KV-Cache Reuse. For KV-cache reuse, VLA-Cache [22] is adopted as the baseline. Its goal is to reduce redundant computation across consecutive control timesteps by reusing cached key/value states for selected visual-token positions. The selected visual tokens remain in the sequence, but their key/value states are reused from the previous control timestep instead of being recomputed from the current-step visual representations. The overall Action-JND-aware KV-cache reuse pipeline is summarized in Alg. 1.

In the original VLA-Cache framework [22], token-reuse decisions are mainly guided by temporal redundancy and task

Algorithm 1 Action-JND-Aware KV-Cache Reuse

Require: Current and previous camera observations I_t and I_{t-1}; Action-JND scores $\mathcal{S}_t = \{s_{t,i}^{\text{rel}}\}_{i=1}^{N}$; previous-step text-to-vision attention scores $\mathcal{A}_{t-1}$; previous KV states $\{K_{t-1}^{(l)}, V_{t-1}^{(l)}\}_{l=1}^{L}$; layer-wise reuse budgets $\{\kappa_t^{(l)}\}_{l=1}^{L}$; reuse policy p.
Ensure: Current KV states $\{K_t^{(l)}, V_t^{(l)}\}_{l=1}^{L}$.

if previous-step attention or KV states are unavailable **then**
  Compute all current-step KV states normally.
  return $\{K_t^{(l)}, V_t^{(l)}\}_{l=1}^{L}$.
end if
Estimate temporal token stability between I_{t-1} and I_t.
Construct the temporally stable token set $\mathcal{T}_t$.
Identify task-relevant protected tokens $\mathcal{A}_t^{\text{prot}}$ using previous-step text-to-vision attention scores $\mathcal{A}_{t-1}$.
Construct safe reuse candidates:
$\mathcal{C}_t^{\text{cache}} = \mathcal{T}_t \setminus \mathcal{A}_t^{\text{prot}}$.
for each cache-controlled decoder layer $l = 1, \ldots, L$ **do**
  Determine the available safe-candidate budget:
  $\bar{\kappa}_t^{(l)} = \min\left(\kappa_t^{(l)}, |\mathcal{C}_t^{\text{cache}}|\right)$.
  Select Action-JND-guided reuse tokens:
  $\mathcal{R}_t^{(l)} = \text{TopK}_{i\in\mathcal{C}_t^{\text{cache}}}\left(s_{t,i}^{\text{rel}}, \bar{\kappa}_t^{(l)}\right)$.
  if $p =$ `strict_guarded` and $|\mathcal{R}_t^{(l)}| < \kappa_t^{(l)}$ **then**
    Construct the guarded fallback pool $\mathcal{B}_t$, excluding the designated high-attention guard positions.
    Fill the remaining reuse budget:
    $\mathcal{R}_t^{(l)} \leftarrow \mathcal{R}_t^{(l)} \cup \text{TopK}_{i\in\mathcal{B}_t\setminus\mathcal{R}_t^{(l)}}\left(s_{t,i}^{\text{rel}}, \kappa_t^{(l)} - |\mathcal{R}_t^{(l)}|\right)$.
  end if
  for each visual token $i \in \mathcal{R}_t^{(l)}$ **do**
    Reuse stale KV states:
    $K_{t,i}^{(l)} \leftarrow K_{t-1,i}^{(l)}, \quad V_{t,i}^{(l)} \leftarrow V_{t-1,i}^{(l)}$.
  end for
  for each visual token $i \notin \mathcal{R}_t^{(l)}$ **do**
    Recompute $K_{t,i}^{(l)}$ and $V_{t,i}^{(l)}$ from the current-step hidden states.
  end for
end for
return $\{K_t^{(l)}, V_t^{(l)}\}_{l=1}^{L}$.

relevance. Specifically, temporally stable tokens are identified by comparing adjacent visual observations, while task-relevant positions are protected according to previous-step text-to-vision attention-based filtering. These cues construct a reuse candidate set $\mathcal{C}_t^{\text{cache}}$, which is shared across decoder layers. As shown in Alg. 1, the Action-JND scores are applied after candidate construction to select reusable token positions. Given a layer-wise reuse budget $\kappa_t^{(l)}$, the number of tokens that can be directly selected from the candidate set is: $\bar{\kappa}_t^{(l)} = \min\left(\kappa_t^{(l)}, |\mathcal{C}_t^{\text{cache}}|\right)$. The candidates are then ranked according to their JND scores, and the initial reuse set at decoder layer l is selected as: $\mathcal{R}_t^{(l)} = \text{TopK}_{i\in\mathcal{C}_t^{\text{cache}}}\left(s_{t,i}^{\text{rel}}, \bar{\kappa}_t^{(l)}\right)$. Since all decoder layers share the same ranking order, the reuse set of each layer corresponds to a different prefix of this common ordering according to the layer-wise reuse schedule.

Under the adaptive `soft` policy, only tokens from the safe candidate set are reused, and the actual reuse ratio may be lower than the requested budget when the candidate set is insufficient. For the setting with a prescribed reuse ratio, the `strict_guarded` policy is used to enforce the target reuse budget. In this setting, when the safe candidate set cannot satisfy the desired reuse budget, the remaining quota is filled from a guarded fallback pool according to the same descending Action-JND ranking strategy, while the designated high-attention positions remain protected from fallback reuse. For each selected token $i \in \mathcal{R}_t^{(l)}$, the current key/value states are replaced by their cached states from the previous timestep:

$$K_{t,i}^{(l)} \leftarrow K_{t-1,i}^{(l)}, \quad V_{t,i}^{(l)} \leftarrow V_{t-1,i}^{(l)}, \quad i \in \mathcal{R}_t^{(l)}. \tag{11}$$

Tokens outside $\mathcal{R}_t^{(l)}$ are recomputed normally from their current-step hidden states. In this way, the original compression policy and schedule of the VLA-Cache pipeline remain unchanged, while the final ranking criterion for safe and fallback candidates is replaced by an action-aware tolerance estimation. Compared with heuristic risk measures, Action-JND encourages action-tolerant tokens to reuse stale KV states while allowing action-sensitive tokens to be recomputed.

JND-Aware Token Pruning. For token pruning, we adapt the safe-candidate construction strategy of VLA-Cache [22] from stale-KV reuse to direct token removal. Visual tokens that are temporally stable and receive low attention during previous action generation are considered potential pruning candidates. Under a fixed pruning budget, the proposed method selects candidates according to their learned Action-JND scores and removes the most action-tolerant ones from the multimodal sequence before subsequent decoder layers. Alg. 2 summarizes the resulting procedure under the `budgeted` policy.

At control step t, let $p_{t,i}$ and $p_{t-1,i}$ denote the aligned RGB patches corresponding to visual token i in the current and previous observations, respectively. Their temporal similarity is computed as $\rho_{t,i} = \cos\left(p_{t,i}, p_{t-1,i}\right)$. To avoid generating an excessively large candidate pool, tokens whose similarity exceeds a threshold τ form an initial stable set. When the number of stable tokens exceeds the prescribed upper bound K_s, only K_s tokens with the highest similarities are retained:

$$\widetilde{\mathcal{T}}_t = \{i \mid \rho_{t,i} \geq \tau\}, \quad \mathcal{T}_t = \text{TopK}_{i\in\widetilde{\mathcal{T}}_t}\left(\rho_{t,i}, \min\left(K_s, |\widetilde{\mathcal{T}}_t|\right)\right). \tag{12}$$

Algorithm 2 Action-JND-Aware Token Pruning

Require: Current and previous observations I_t and I_{t-1}; previous-step action-to-vision attention scores a_{t-1}^{act}; Action-JND scores $\mathcal{S}_t = \{s_{t,i}^{\text{abs}}\}_{i=1}^N$; similarity threshold τ; stable-token limit K_s; protection budget K_a; pruning budget κ_t.
Ensure: Retained visual-token sequence F_t^{keep}.
Let $\mathcal{V} = \{1, \ldots, N\}$ denote all visual-token positions.
if previous-step guidance is unavailable **then**
  return F_t.
end if
Compute aligned RGB-patch similarities $\{\rho_{t,i}\}_{i=1}^N$.
Construct the thresholded stable set:
  $\widetilde{\mathcal{T}}_t = \{i \mid \rho_{t,i} \geq \tau\}$.
Retain at most K_s stable tokens:
  $\mathcal{T}_t = \text{TopK}_{i\in\widetilde{\mathcal{T}}_t}\left(\rho_{t,i}, \min\left(K_s, |\widetilde{\mathcal{T}}_t|\right)\right)$.
Construct the action-attention protection set:
  $\mathcal{A}_t^{\text{prot}} = \text{TopK}\left(a_{t-1}^{\text{act}}, K_a\right)$.
Construct safe pruning candidates:
  $\mathcal{C}_t^{\text{prune}} = \mathcal{T}_t \setminus \mathcal{A}_t^{\text{prot}}$.
Select safe candidates using descending Action-JND scores:
  $\mathcal{P}_t = \text{TopK}_{i\in\mathcal{C}_t^{\text{prune}}}\left(s_{t,i}^{\text{abs}}, \min\left(\kappa_t, |\mathcal{C}_t^{\text{prune}}|\right)\right)$.
if $|\mathcal{P}_t| < \kappa_t$ **then**
  Fill the remaining pruning budget:
  $\mathcal{P}_t \leftarrow \mathcal{P}_t \cup \text{TopK}_{i\in\mathcal{V}\setminus\mathcal{P}_t}\left(s_{t,i}^{\text{abs}}, \kappa_t - |\mathcal{P}_t|\right)$.
end if
Physically remove the selected visual tokens:
  $F_t^{\text{keep}} = [f_{t,i} \mid i \notin \mathcal{P}_t]$.
Feed F_t^{keep} into decoder layer index 3 and subsequent layers.
return F_t^{keep}.

To protect tokens associated with action generation process, the K_a positions receiving the highest action-to-vision attention at the previous control timestep are collected as $\mathcal{A}_t^{\text{prot}} = \text{TopK}\left(a_{t-1}^{\text{act}}, K_a\right)$. Since token pruning is performed at an intermediate layer during prefilling, we record the previous-timestep action-to-vision attention from layers preceding the pruning layer during action decoding, where the complete N-token visual grid remains available. The safe pruning candidate pool is then constructed as $\mathcal{C}_t^{\text{prune}} = \mathcal{T}_t \setminus \mathcal{A}_t^{\text{prot}}$, where $\setminus$ means token removal. Let K_t denote the target number of retained tokens, and $\kappa_t = N - K_t$ the corresponding number of tokens to be removed. The candidate tokens are ranked in descending order of $s_{t,i}^{\text{abs}}$, such that tokens with greater action tolerance are removed first. Under the `budgeted` policy, which enforces a fixed token budget, the final pruning set is,

$$\mathcal{P}_t = \begin{cases} \text{TopK}_{i\in\mathcal{C}_t^{\text{prune}}}\left(s_{t,i}^{\text{abs}}, \kappa_t\right), & |\mathcal{C}_t^{\text{prune}}| \geq \kappa_t, \\ \mathcal{C}_t^{\text{prune}} \cup \text{TopK}_{i\notin\mathcal{C}_t^{\text{prune}}}\left(s_{t,i}^{\text{abs}}, \kappa_t - |\mathcal{C}_t^{\text{prune}}|\right), & |\mathcal{C}_t^{\text{prune}}| < \kappa_t. \end{cases} \tag{13}$$

This formulation ensures that the safe candidate pool is always prioritized. When the candidate pool contains fewer than κ_t tokens, all safe candidates are pruned first, and the remaining pruning quota is filled from the other visual-token positions according to the same descending Action-JND ranking. Although these additional pruning candidates may fall outside the original protection constraints, selecting them according to Action-JND still prioritizes tokens with relatively high action tolerance, thereby reducing the risk of affecting the downstream robot action.

TABLE I

PERFORMANCE OF ACTION-JND AND KV-CACHE BASELINES ON OPENVLA ON THE LIBERO BENCHMARK [50]. VANILLA OPENVLA SERVES AS THE UNCOMPRESSED REFERENCE. FOR EACH REUSE SETTING, SUCCESS RATES (%) ON *Spatial*, *Object*, *Goal*, AND *Long* AND THEIR MEAN (*Acc.*) ARE REPORTED. ΔACC. DENOTES THE ACCURACY DIFFERENCE RELATIVE TO VLA-CACHE [22] UNDER THE SAME SETTING. FLOPS, RELATIVE FLOPS, CUDA LATENCY, LATENCY DIFFERENCE RELATIVE TO VANILLA OPENVLA, AND CONTROL FREQUENCY ARE ALSO REPORTED. BLUE AND RED INDICATE THE BEST AND WORST RESULTS, RESPECTIVELY, WITHIN EACH REUSE SETTING ACCORDING TO THE INDICATED ↑ / ↓ DIRECTION, WHILE PINK ROWS HIGHLIGHT ACTION-JND

Method	OpenVLA										
	Spatial	Object	Goal	Long	Acc.(%)↑	ΔAcc.↑	FLOPs(T)↓	Rel. FLOPs(%)↓	Latency(ms)↓	ΔLatency(ms)↓	Freq.(Hz)↑
Upper Bound (Vanilla, 100%)											
Vanilla	81.40	64.40	79.20	57.20	**70.55**	–	1.864	100.00%	46.07	–	21.71
Soft Adaptive KV Cache											
FastV [6]	79.80	60.20	74.60	48.00	**65.65**	**+1.55**	1.446	77.58%	40.95	-5.12	24.42
SparseVLM [7]	77.80	60.20	76.40	48.80	**65.80**	**+1.70**	1.446	77.59%	41.57	-4.50	24.06
DivPrune [8]	78.60	61.80	75.20	48.20	**65.95**	**+1.85**	1.434	76.93%	43.20	-2.88	23.19
VLA-Cache [22]	77.60	58.20	75.40	45.20	**64.10**	**+0.00**	1.431	76.75%	41.68	-4.39	24.02
Action-JND (Ours)	79.60	64.40	74.40	50.00	**67.10**	**+3.00**	1.445	77.52%	40.71	-5.36	24.57
KV Cache Reuse Ratio = 30%											
FastV [6]	77.60	60.40	74.00	46.00	**64.50**	**-0.20**	1.339	71.83%	43.79	-2.29	22.86
SparseVLM [7]	77.60	61.40	73.40	47.00	**64.85**	**+0.15**	1.339	71.84%	47.15	+1.07	21.25
DivPrune [8]	77.60	61.60	76.80	45.60	**65.40**	**+0.70**	1.339	71.85%	39.73	-6.34	25.19
VLA-Cache [22]	77.40	60.20	76.00	45.20	**64.70**	**+0.00**	1.339	71.84%	39.85	-6.23	25.11
Action-JND (Ours)	80.80	62.20	75.60	49.80	**67.10**	**+2.40**	1.340	71.90%	38.55	-7.52	25.95
KV Cache Reuse Ratio = 40%											
FastV [6]	76.00	60.60	73.60	46.40	**64.15**	**-1.05**	1.158	62.10%	42.86	-3.21	23.34
SparseVLM [7]	76.00	58.40	73.40	43.40	**62.80**	**-2.40**	1.158	62.10%	37.33	-8.74	26.79
DivPrune [8]	77.00	60.20	76.80	46.60	**65.15**	**-0.05**	1.162	62.32%	36.66	-9.41	27.28
VLA-Cache [22]	78.60	62.40	74.20	45.60	**65.20**	**+0.00**	1.160	62.24%	37.85	-8.22	26.42
Action-JND (Ours)	77.80	62.20	78.60	47.60	**66.55**	**+1.35**	1.162	62.33%	36.20	-9.87	27.62
KV Cache Reuse Ratio = 60%											
FastV [6]	41.20	41.60	64.20	10.20	**39.30**	**+6.10**	0.785	42.13%	31.54	-14.53	31.71
SparseVLM [7]	38.60	37.60	64.60	12.80	**38.40**	**+5.20**	0.785	42.13%	31.87	-14.20	31.39
DivPrune [8]	40.20	49.00	70.60	17.00	**44.20**	**+11.00**	0.791	42.44%	31.88	-14.19	31.38
VLA-Cache [22]	31.40	30.60	63.20	7.60	**33.20**	**+0.00**	0.792	42.49%	32.36	-13.71	30.91
Action-JND (Ours)	71.20	60.80	71.00	24.60	**56.90**	**+23.70**	0.793	42.54%	30.18	-15.89	33.14
KV Cache Reuse Ratio = 80%											
FastV [6]	32.40	33.80	61.00	8.00	**33.80**	**+20.00**	0.742	39.83%	43.34	-2.73	23.19
SparseVLM [7]	33.60	35.80	61.80	10.40	**35.40**	**+21.60**	0.743	39.84%	43.69	-2.38	22.91
DivPrune [8]	31.00	40.80	63.20	14.60	**37.40**	**+23.60**	0.745	39.96%	31.13	-14.94	32.13
VLA-Cache [22]	10.00	7.60	35.80	1.80	**13.80**	**+0.00**	0.749	40.21%	31.69	-14.38	31.56
Action-JND (Ours)	69.40	58.80	69.20	24.40	**55.45**	**+41.65**	0.754	40.44%	34.79	-11.28	28.81

V. EXPERIMENTS

A. Experimental Setup

Benchmarks and backbones. Experiments are conducted on the LIBERO benchmark [50], a commonly used benchmark for evaluating language-conditioned robotic manipulation and VLA policy learning. Following standard VLA evaluation protocols [20], [22], [28], four standard LIBERO suites are used for evaluation: *Spatial*, *Object*, *Goal*, and *Long*, which cover spatial reasoning, object-centric manipulation, goal-conditioned execution, and long-horizon tasks, respectively. The proposed Action-JND is evaluated on two representative VLA backbones: OpenVLA [19] and OpenVLA-OFT [20]. OpenVLA uses a single RGB view and predicts discretized action tokens. OpenVLA-OFT predicts continuous action chunks and can take both primary-view and wrist-view observations.

Training configuration. For OpenVLA, the JND estimator is optimized with the action-token KL objective in Eq. (4), using $\lambda_{\text{act}} = 1.0$ and $\lambda_{\text{mag}} \in \{0.02, 0.01, 0.005\}$. For OpenVLA-OFT, the primary- and wrist-view estimators are trained separately with the continuous action-regression objective in Eq. (5), perturbing one view while keeping the other clean. We set $\lambda_{\text{act}} = 1.0$, $\lambda_{\text{mag}} = 3 \times 10^{-4}$ for primary view, while sweeping $\lambda_{\text{mag}} \in \{3, 5, 7, 8\} \times 10^{-4}$ for wrist view.

Baselines. For KV-cache reuse, we compare Action-JND with VLA-Cache [22] and KV-cache-compatible adaptations of FastV [6], SparseVLM [7], and DivPrune [8]. For token pruning, we compare Action-JND with FastV [6], SparseVLM [7], DivPrune [8], and a VLA-Cache-based pruning baseline [22], which transfers the candidate construction and proxy-risk ranking of VLA-Cache to visual-token pruning.

Compression settings. For KV-cache reuse, we evaluate the adaptive `soft` policy, whose actual reuse ratio depends on the available safe candidates, and the `strict_guarded` policy with target reuse ratios of 30%, 40%, 60%, and 80%. For token pruning, we evaluate the `budgeted` policy with target pruning ratios of 25%, 50%, 75%, and 87.5%.

Implementation environments. The KV-cache experiments are built on the publicly released VLA-Cache [22] codebase, whereas the token-pruning-related experiments are built upon the official OpenVLA codebase [19]. Since the two codebases rely on different software dependencies, the reported vanilla OpenVLA results of different compression tasks may differ.

Evaluation metrics. Task performance is evaluated using the success rates on each LIBERO suite and their mean, denoted as average accuracy (*Acc.*). ΔAcc. measures the accuracy difference relative to the corresponding baseline under the same compression setting. For efficiency, FLOPs quantify the theoretical computational cost, while relative FLOPs report the

TABLE II

PERFORMANCE OF ACTION-JND AND KV-CACHE BASELINES ON OPENVLA-OFT ON THE LIBERO BENCHMARK [50]. VANILLA OPENVLA-OFT SERVES AS THE UNCOMPRESSED REFERENCE. FOR EACH REUSE SETTING, SUITE-WISE AND AVERAGE SUCCESS RATES ARE REPORTED, WITH ΔACC. DENOTING THE ACCURACY DIFFERENCE FROM VLA-CACHE [22] UNDER THE SAME SETTING. FLOPS, RELATIVE FLOPS, CUDA LATENCY, LATENCY DIFFERENCE FROM VANILLA OPENVLA-OFT, AND CONTROL FREQUENCY ARE ALSO REPORTED. THE COLOR ANNOTATIONS FOLLOW TABLE I

Method	OpenVLA-OFT										
	Spatial	Object	Goal	Long	Acc.(%)↑	ΔAcc.↑	FLOPs(T)↓	Rel. FLOPs(%)↓	Latency(ms)↓	ΔLatency(ms)↓	Freq.(Hz)↑
Upper Bound (Vanilla, 100%)											
Vanilla	94.20	97.60	95.80	92.60	**95.05**	–	3.970	100.00%	137.43	–	58.23
Soft Adaptive KV Cache											
FastV [6]	94.20	97.60	96.80	93.20	**95.45**	**+0.10**	3.244	81.72%	132.30	-5.13	60.51
SparseVLM [7]	94.40	97.40	96.40	93.00	**95.30**	**-0.05**	3.245	81.73%	139.58	+2.15	57.40
DivPrune [8]	95.00	97.40	96.00	93.80	**95.55**	**+0.20**	3.251	81.88%	135.04	-2.39	59.27
VLA-Cache [22]	94.40	97.80	95.60	93.60	**95.35**	**+0.00**	3.239	81.59%	106.44	-30.99	75.19
Action-JND (Ours)	94.60	97.60	97.60	95.00	**96.20**	**+0.85**	3.242	81.67%	111.63	-25.80	72.09
KV Cache Reuse Ratio = 30%											
FastV [6]	94.60	97.60	95.60	93.00	**95.20**	**-0.25**	2.991	75.35%	126.46	-10.97	63.27
SparseVLM [7]	93.60	97.20	97.00	94.40	**95.55**	**+0.10**	2.991	75.35%	127.29	-10.13	62.86
DivPrune [8]	94.40	98.40	97.40	91.80	**95.50**	**+0.05**	2.992	75.37%	130.14	-7.29	61.47
VLA-Cache [22]	93.60	98.40	96.80	93.00	**95.45**	**+0.00**	2.991	75.34%	121.26	-16.17	65.99
Action-JND (Ours)	94.20	98.00	96.80	93.00	**95.50**	**+0.05**	2.992	75.37%	106.83	-30.60	75.46
KV Cache Reuse Ratio = 40%											
FastV [6]	94.60	97.40	96.80	92.20	**95.25**	**-0.50**	2.661	67.04%	122.22	-15.21	65.51
SparseVLM [7]	94.20	97.00	96.40	92.60	**95.05**	**-0.70**	2.661	67.04%	131.86	-5.57	60.67
DivPrune [8]	93.80	97.60	96.80	93.40	**95.40**	**-0.35**	2.662	67.06%	118.96	-18.46	67.31
VLA-Cache [22]	94.00	98.20	97.20	93.60	**95.75**	**+0.00**	2.661	67.03%	112.75	-24.68	70.96
Action-JND (Ours)	94.20	97.60	96.40	92.40	**95.15**	**-0.60**	2.664	67.09%	100.91	-36.52	80.37
KV Cache Reuse Ratio = 60%											
FastV [6]	89.40	80.80	97.20	77.60	**86.25**	**-0.25**	2.001	50.40%	108.14	-29.29	75.51
SparseVLM [7]	90.00	81.20	96.00	77.20	**86.10**	**-0.40**	2.001	50.41%	109.68	-27.75	73.12
DivPrune [8]	90.60	92.80	96.00	85.20	**91.15**	**+4.65**	2.006	50.53%	106.81	-30.62	75.16
VLA-Cache [22]	89.00	88.20	94.40	74.40	**86.50**	**+0.00**	1.993	50.20%	100.38	-37.05	79.93
Action-JND (Ours)	92.00	96.80	97.40	84.60	**92.70**	**+6.20**	2.010	50.63%	90.45	-46.98	90.53
KV Cache Reuse Ratio = 80%											
FastV [6]	89.20	76.00	95.40	76.00	**84.15**	**+4.70**	1.974	49.72%	108.17	-29.26	75.61
SparseVLM [7]	90.20	76.20	95.60	78.00	**85.00**	**+5.55**	1.974	49.73%	113.61	-23.82	70.96
DivPrune [8]	90.80	81.40	95.60	83.40	**87.80**	**+8.35**	1.981	49.91%	107.54	-29.89	74.72
VLA-Cache [22]	88.80	66.20	92.80	70.00	**79.45**	**+0.00**	1.962	49.42%	100.10	-37.33	79.94
Action-JND (Ours)	92.00	78.20	97.60	90.40	**89.55**	**+10.10**	1.963	49.45%	91.05	-46.38	89.45

remaining computation as a percentage of the vanilla model. CUDA latency measures the actual GPU execution time for action generation, ΔLatency denotes its difference relative to the vanilla model, and control frequency measures how frequently the policy can update robot actions during closed-loop control. A higher control frequency enables more frequent action updates during closed-loop execution.

B. Experimental Analysis

Here we analyze the effectiveness of Action-JND under KV-cache reuse and token pruning, focusing on task preservation, inference efficiency, and qualitative compression behavior.

1) Result Analysis on KV-Cache Reuse: Tables I and II show that Action-JND provides more reliable KV-cache reuse on both OpenVLA and OpenVLA-OFT backbones. Under both `soft` and `strict_guarded` reuse policies, Action-JND consistently achieves competitive or superior task performance across different reuse ratios while maintaining computational costs comparable to those of existing methods. Notably, it achieves the highest average accuracy under the `soft` policy on both backbones and generally exhibits a more favorable accuracy–efficiency trade-off under the `strict_guarded` policy. At relatively low reuse ratios, performance differences among the compared methods are modest because only a conservative subset of visual representations is reused. As the reuse ratio increases, however, the advantage of Action-JND becomes increasingly pronounced, whereas methods based on manually designed compression criteria suffer rapid performance degradation.

Existing methods mainly determine cache reuse using indirect compression cues such as attention, language relevance, visual redundancy, or proxy risk. In contrast, Action-JND directly estimates the action tolerance of each visual token and prioritizes stale-KV reuse for tokens whose representation changes are less likely to alter the predicted action, thereby better preserving action-critical information. At 60% and 80% reuse ratios, Action-JND improves the average accuracy over VLA-Cache [22] by 23.70 and 41.65 percentage points on OpenVLA, and by 6.20 and 10.10 percentage points on OpenVLA-OFT, respectively. Under matched reuse ratios and comparable FLOPs, these gains demonstrate the effectiveness of the proposed action-conditioned tolerance criterion in identifying tokens suitable for stale-KV reuse. Action-JND also reduces CUDA latency and increases control frequency relative to vanilla inference, enabling more frequent action updates while better preserving task performance under aggressive KV-cache reuse.

2) Result Analysis on Token Pruning: Table III shows that Action-JND provides more reliable visual-token pruning on OpenVLA. From 50% pruning onward, Action-JND consistently achieves the highest average accuracy among all compared schemes [6]–[8], [22], with its advantage becoming more pronounced as the pruning ratio increases. Specifically, compared with the VLA-Cache-based pruning baseline [22],

TABLE III

PERFORMANCE OF ACTION-JND AND TOKEN-PRUNING BASELINES ON OPENVLA ON THE LIBERO BENCHMARK [50]. VANILLA OPENVLA SERVES AS THE UNCOMPRESSED REFERENCE. FOR EACH PRUNING SETTING, SUITE-WISE AND AVERAGE SUCCESS RATES ARE REPORTED, WITH ΔACC. DENOTING THE ACCURACY DIFFERENCE RELATIVE TO THE VLA-CACHE-BASED PRUNING BASELINE [22] UNDER THE SAME SETTING. FLOPS, RELATIVE FLOPS, CUDA LATENCY, LATENCY DIFFERENCE RELATIVE TO VANILLA OPENVLA, AND CONTROL FREQUENCY ARE ALSO REPORTED. THE COLOR ANNOTATIONS FOLLOW TABLE I

Method	OpenVLA										
	Spatial	Object	Goal	Long	Acc.(%)↑	ΔAcc.↑	FLOPs(T)↓	Rel. FLOPs(%)↓	Latency(ms)↓	ΔLatency(ms)↓	Freq.(Hz)↑
Upper Bound (Vanilla, 0% Pruned)											
Vanilla	79.80	61.40	72.60	51.80	**66.40**	–	1.985	100.00%	52.88	–	18.91
Token Pruning Ratio = 25%											
FastV [6]	81.20	59.60	75.00	49.80	**66.40**	**-1.94**	1.593	80.26%	40.46	-12.43	24.72
SparseVLM [7]	80.20	57.40	74.60	46.80	**64.75**	**-3.59**	1.593	80.26%	41.40	-11.49	24.16
DivPrune [8]	77.40	37.20	74.40	36.20	**56.30**	**-12.04**	1.553	78.25%	37.06	-15.82	26.98
VLA-Cache [22]	80.30	63.80	75.60	53.60	**68.34**	**+0.00**	1.593	80.26%	40.53	-12.36	24.67
Action-JND (Ours)	81.60	62.80	77.60	47.20	**67.30**	**-1.04**	1.628	81.99%	43.67	-9.22	22.90
Token Pruning Ratio = 50%											
FastV [6]	81.40	55.40	76.40	48.20	**65.35**	**-1.20**	1.203	60.61%	36.58	-16.31	27.34
SparseVLM [7]	81.60	58.20	72.40	47.80	**65.00**	**-1.55**	1.203	60.61%	39.06	-13.82	25.60
DivPrune [8]	46.60	35.20	66.20	16.40	**41.10**	**-25.45**	1.124	56.62%	33.19	-19.70	30.13
VLA-Cache [22]	80.40	60.00	75.20	50.60	**66.55**	**+0.00**	1.203	60.61%	34.59	-18.30	28.91
Action-JND (Ours)	84.00	62.00	76.60	49.40	**68.00**	**+1.45**	1.238	62.34%	37.74	-15.15	26.50
Token Pruning Ratio = 75%											
FastV [6]	79.40	45.00	70.20	33.60	**57.05**	**-0.80**	0.815	41.07%	32.05	-20.84	31.20
SparseVLM [7]	79.40	43.80	70.60	33.00	**56.70**	**-1.15**	0.815	41.07%	33.52	-19.36	29.83
DivPrune [8]	6.60	32.40	39.60	4.80	**20.85**	**-37.00**	0.697	35.09%	26.16	-26.72	38.22
VLA-Cache [22]	78.00	43.00	68.60	41.80	**57.85**	**+0.00**	0.815	41.07%	31.02	-21.87	32.24
Action-JND (Ours)	76.40	51.00	71.00	45.20	**60.90**	**+3.05**	0.850	42.80%	31.05	-21.84	32.21
Token Pruning Ratio = 87.5%											
FastV [6]	55.40	14.60	56.60	15.20	**35.45**	**+5.10**	0.622	31.33%	28.95	-23.94	34.55
SparseVLM [7]	60.40	7.40	56.00	13.80	**34.40**	**+4.05**	0.622	31.33%	28.48	-24.41	35.12
DivPrune [8]	2.60	20.20	19.00	0.60	**10.60**	**-19.75**	0.484	24.36%	26.02	-26.86	38.43
VLA-Cache [22]	46.20	12.40	49.80	13.00	**30.35**	**+0.00**	0.622	31.33%	30.32	-22.56	32.98
Action-JND (Ours)	65.40	22.60	57.00	23.00	**42.00**	**+11.65**	0.656	33.06%	30.16	-22.72	33.15

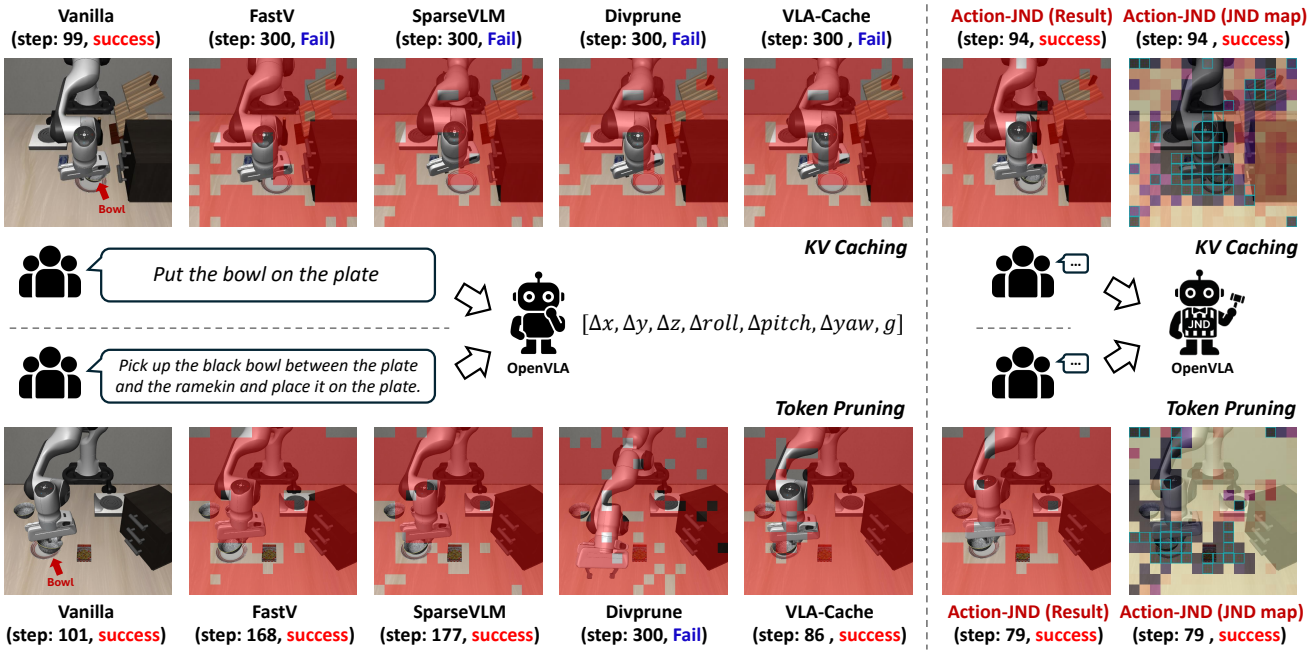


Fig. 5. **Qualitative visualization of different token-compression baselines and Action-JND-aware token compression.** The upper and lower rows show representative results for KV-cache reuse and token pruning on LIBERO, respectively. The red overlays indicate visual-token regions selected for stale-KV reuse or physical pruning. The last two columns show the Action-JND compression result and its corresponding JND heatmap. Each heatmap visualizes the tolerance of individual visual tokens to feature perturbations. Color transitions from dark purple/black to red, orange, and bright yellow, where brighter colors indicate higher JND values and stronger action tolerance, making the corresponding tokens more suitable for reuse or pruning. In contrast, darker colors indicate action-sensitive regions that should be preserved or recomputed. The blue boxes in the JND heatmaps indicate the visual-token positions finally selected for stale-KV reuse or pruning. The step count denotes the number of control timesteps required for task execution, together with the final success or failure status.

Action-JND improves the average accuracy by 1.45, 3.05, and 11.65 percentage points at pruning ratios of 50%, 75%, and 87.5%, respectively. It also consistently outperforms the other methods based on indirect pruning cues; even at 87.5% pruning, it exceeds the strongest competing method by 6.55 percentage points. These results suggest that compression criteria based on indirect cues become increasingly unreliable under aggressive pruning. Unlike KV-cache reuse, token

pruning physically removes visual representations from all subsequent layers, making errors caused by removing action-sensitive tokens difficult to recover from in subsequent layers.

In contrast, Action-JND directly estimates the action tolerance of each visual token and removes tokens whose absence is less likely to affect the predicted action, thereby better preserving manipulation-critical information. At the aggressive pruning ratio of 87.5%, Action-JND maintains an average accuracy of 42.00%, compared with 30.35% for the VLA-Cache-based pruning baseline and 35.45% for the strongest competing baseline. Meanwhile, it requires only 33.06% of the FLOPs of the vanilla model, while reducing CUDA latency from 52.88 ms to 30.16 ms and increasing control frequency from 18.91 to 33.15 Hz. These results demonstrate that Action-JND substantially reduces inference cost while retaining good task performance, providing a favorable trade-off between action prediction and inference efficiency.

3) Qualitative Visualization: Fig. 5 presents qualitative comparisons of different token-compression baselines and Action-JND-aware token compression for KV-cache reuse and token pruning under aggressive compression. These examples show that Action-JND better protects action-sensitive information, enabling successful task execution in cases where several competing methods fail or require more control steps.

VI. Conclusion

In this paper, we put forward JND modeling toward embodied perception, and introduce Action-JND to characterize the maximum visual-token variation tolerable to a VLA policy without noticeably changing its action response. We further instantiate this formulation with a lightweight token-wise JND estimator and apply the learned action-tolerance scores to two representative token-compression paradigms, including KV-cache reuse and token pruning. Experiments on LIBERO with OpenVLA and OpenVLA-OFT demonstrate that Action-JND enables more reliable compression, particularly under aggressive compression ratios, while improving inference efficiency for closed-loop robot control. In future work, we will extend embodied JND modeling to more diverse applications and tasks, including vision-language navigation with quadruped robots and coordinated multi-platform manipulation, and investigate its generalization across diverse embodiments and action spaces.

References

[1] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.

[2] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 21 875–21 895.

[3] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "SnapKV: LLM knows what you are looking for before generation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 22 947–22 970, 2024.

[4] Z. Cai, Y. Zhang, B. Gao, Y. Liu, Y. Li, T. Liu, K. Lu, W. Xiong, Y. Dong, J. Hu *et al.*, "Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling," *arXiv preprint arXiv:2406.02069*, 2024.

[5] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman, "Token merging: Your vit but faster," *arXiv preprint arXiv:2210.09461*, 2022.

[6] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, C. Zhou, and B. Chang, "An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models," in *European Conference on Computer Vision*. Springer, 2024, pp. 19–35.

[7] Y. Zhang, C.-K. Fan, J. Ma, W. Zheng, T. Huang, K. Cheng, D. Gudovskiy, T. Okuno, Y. Nakata, K. Keutzer *et al.*, "SparseVLM: Visual token sparsification for efficient vision-language model inference," *arXiv preprint arXiv:2410.04417*, 2024.

[8] S. R. Alvar, G. Singh, M. Akbari, and Y. Zhang, "DivPrune: Diversity-based visual token pruning for large multimodal models," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 9392–9401.

[9] S. Yang, Y. Chen, Z. Tian, C. Wang, J. Li, B. Yu, and J. Jia, "Visionzip: Longer is better but not necessary in vision language models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 19 792–19 802.

[10] Y. Shang, M. Cai, B. Xu, Y. J. Lee, and Y. Yan, "Llava-prumerge: Adaptive token reduction for efficient large multimodal models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 22 857–22 867.

[11] C. Gao, Y. Jiang, L. Li, D. Liu, and F. Wu, "DMOFC: Discrimination metric-optimized feature compression," *arXiv preprint arXiv:2405.04044*, 2024.

[12] C. Gao, Z. Liu, L. Li, D. Liu, X. Sun, and W. Lin, "DT-UFC: Universal large model feature coding via peaky-to-balanced distribution transformation," in *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025, pp. 5198–5207.

[13] Y. Huang, Z. Wu, L. Wang, and T. Tan, "Feature coding in image classification: A comprehensive study," *IEEE transactions on pattern analysis and machine intelligence*, vol. 36, no. 3, pp. 493–506, 2013.

[14] C. Gao, Y. Ma, Q. Chen, Y. Xu, D. Liu, and W. Lin, "Feature coding in the era of large models: Dataset, test conditions, and benchmark," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 1068–1077.

[15] C. Li, J. Xiao, J. Zhang, F. Wen, Z. Zhang, Y. Tian, X. Zhu, X. Liu, Z. Cheng, W. Lin *et al.*, "Image quality assessment for embodied ai," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 39 960–39 982.

[16] C. Li, R. Qing, J. Zhang, Y. Tian, X. Zhu, Z. Zhang, X. Liu, W. Lin, and G. Zhai, "Embodied image compression," *arXiv preprint arXiv:2512.11612*, 2025.

[17] C. Gao, W. Zhou, G. Lin, and W. Lin, "Compressed feature quality assessment: Dataset and baselines," in *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025, pp. 13 450–13 456.

[18] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.

[19] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[20] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025.

[21] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.

[22] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang, and C. Xu, "VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching," *Advances in Neural Information Processing Systems*, vol. 38, pp. 164 448–164 473, 2026.

[23] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, Z. Zhang, C. Wen, and L. Zhang, "EfficientVLA: Training-free acceleration and compression for vision-language-action models," *Advances in Neural Information Processing Systems*, vol. 38, pp. 40 891–40 914, 2026.

[24] Y. Li, Y. Meng, Z. Sun, K. Ji, C. Tang, J. Fan, X. Ma, S. Xia, Z. Wang, and W. Zhu, "SP-VLA: A joint model scheduling and token pruning approach for vla model acceleration," *arXiv preprint arXiv:2506.12723*, 2025.

[25] H. Wang, J. Xu, Y. Xiang, J. Pan, Y. Zhou, Y.-L. Li, and G. Dai, "SpecPrune-VLA: Accelerating vision-language-action models via action-aware self-speculative pruning," *arXiv preprint arXiv:2509.05614*, 2025.

[26] T. Jiang, X. Jiang, Y. Ma, X. Wen, B. Li, K. Zhan, P. Jia, Y. Liu, S. Sun, and X. Lang, "The better you learn, the smarter you prune: Towards efficient vision-language-action models via differentiable token pruning," *arXiv preprint arXiv:2509.12594*, 2025.
[27] X. Pei, Y. Chen, S. Xu, Y. Wang, Y. Shi, and C. Xu, "Action-aware dynamic pruning for efficient vision-language-action manipulation," *arXiv preprint arXiv:2509.22093*, 2025.
[28] Z. Liu, Y. Chen, H. Cai, T. Lin, S. Yang, Z. Liu, and B. Zhao, "VLA-Pruner: Temporal-aware dual-level visual token pruning for efficient vision-language-action inference," *arXiv preprint arXiv:2511.16449*, 2025.
[29] H. Fang, Y. Liu, Y. Du, L. Du, and H. Yang, "SQAP-VLA: A synergistic quantization-aware pruning framework for high-performance vision-language-action models," *arXiv preprint arXiv:2509.09090*, 2025.
[30] G. J. Sullivan and T. Wiegand, "Rate-distortion optimization for video compression," *IEEE signal processing magazine*, vol. 15, no. 6, pp. 74–90, 1998.
[31] B. Bross, Y.-K. Wang, Y. Ye, S. Liu, J. Chen, G. J. Sullivan, and J.-R. Ohm, "Overview of the versatile video coding (VVC) standard and its applications," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 10, pp. 3736–3764, 2021.
[32] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (hevc) standard," *IEEE Transactions on circuits and systems for video technology*, vol. 22, no. 12, pp. 1649–1668, 2012.
[33] Z. Li, J. Liao, C. Tang, H. Zhang, Y. Li, Y. Bian, X. Sheng, X. Feng, Y. Li, C. Gao *et al.*, "USTC-TD: A test dataset and benchmark for image and video coding in 2020s," *IEEE Transactions on Multimedia*, 2025.
[34] C.-H. Chou and Y.-C. Li, "A perceptually tuned subband image coder based on the measure of just-noticeable-distortion profile," *IEEE Transactions on circuits and systems for video technology*, vol. 5, no. 6, pp. 467–476, 1995.
[35] N. Jayant, J. Johnston, and R. Safranek, "Signal compression based on models of human perception," *Proceedings of the IEEE*, vol. 81, no. 10, pp. 1385–1422, 2002.
[36] W. Lin and G. Ghinea, "Progress and opportunities in modelling just-noticeable difference (jnd) for multimedia," *IEEE transactions on multimedia*, vol. 24, pp. 3706–3721, 2021.
[37] J. Wu, G. Shi, W. Lin, A. Liu, and F. Qi, "Just noticeable difference estimation for images with free-energy principle," *IEEE Transactions on Multimedia*, vol. 15, no. 7, pp. 1705–1710, 2013.
[38] J. Jin, X. Zhang, X. Fu, H. Zhang, W. Lin, J. Lou, and Y. Zhao, "Just noticeable difference for deep machine vision," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 32, no. 6, pp. 3452–3461, 2021.
[39] Q. Zhang, S. Wang, X. Zhang, S. Ma, and W. Gao, "Just recognizable distortion for machine vision oriented image and video coding," *International Journal of Computer Vision*, vol. 129, no. 10, pp. 2889–2906, 2021.
[40] Q. Zhang, S. Wang, X. Zhang, C. Jia, Z. Wang, S. Ma, and W. Gao, "Perceptual video coding for machines via satisfied machine ratio modeling," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 7651–7668, 2024.
[41] X. Shen, H. Ou, Z. Ni, W. Yang, S. Wang, and S. Kwong, "Transferring from distortion to perception-oriented optimization: Just-noticeable-distortion-based domain adaptation," *IEEE Transactions on Multimedia*, 2025.
[42] C. Gao, D. Liu, L. Li, and F. Wu, "Towards task-generic image compression: A study of semantics-oriented metrics," *IEEE Transactions on Multimedia*, vol. 25, pp. 721–735, 2021.
[43] Z. Chen, Y. Tian, Y. Sun, W. Sun, Z. Zhang, W. Lin, G. Zhai, and W. Zhang, "Just noticeable difference for large multimodal models," *arXiv preprint arXiv:2507.00490*, 2025.
[44] R. Zhao, W. Li, L. Zhu, Y. Zheng, and W. Lin, "Just noticeable difference modeling for deep visual features," in *International Conference on Machine Learning*, 2026.
[45] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.
[46] H. Lin, G. Chen, M. Jenadeleh, V. Hosu, U.-D. Reips, R. Hamzaoui, and D. Saupe, "Large-scale crowdsourced subjective assessment of picturewise just noticeable difference," *IEEE transactions on circuits and systems for video technology*, vol. 32, no. 9, pp. 5859–5873, 2022.
[47] G. Chen, H. Lin, O. Wiedemann, and D. Saupe, "Localization of just noticeable difference for image compression," in *2023 15th International Conference on Quality of Multimedia Experience (QoMEX)*. IEEE, 2023, pp. 61–66.
[48] Q. Jiang, Z. Liu, S. Wang, F. Shao, and W. Lin, "Toward top-down just noticeable difference estimation of natural images," *IEEE Transactions on Image Processing*, vol. 31, pp. 3697–3712, 2022.
[49] M. Wang, Y. Zhu, R. Zhang, and W. Xie, "MetaJND: A meta-learning approach for just noticeable difference estimation." in *IJCAI*, 2024, pp. 3151–3159.
[50] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.

Zhuoyuan Li (Member, IEEE) received the B.S. degree in communication engineering from Southwest Jiaotong University, China, in 2020, and the Ph.D. degree in electronic engineering and information science from the University of Science and Technology of China (USTC), China, in 2025. He is currently a Postdoctoral Researcher at The Hong Kong Polytechnic University (PolyU), supervised by Kenneth Kin-man Lam. His research interests include image and video coding. He has published over 30 papers in international journals and conferences. He won several technical challenges at ICIP 2024, MMSP 2024, and VCIP 2025. He received the PRCV 2025 Grand Challenge Outstanding Exploration Award and the NeurIPS 2025 Top Reviewer Award. He has submitted several technical proposals to ISO/IEC and AVS standardization activities.

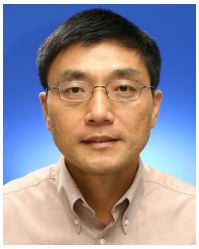
Rui Zhao (Member, IEEE) received the B.E. degree in Communication Engineering from Tianjin University, Tianjin, China, in 2020, and the Ph.D. degree in Computer Science from Peking University, Beijing, China, in 2025. He is currently a postdoctoral research fellow with Nanyang Technological University, Singapore. His research interests include computational photography, computer vision, and image processing, particularly for topics related to motion estimation and neuromorphic cameras.

Jin Wang received the M.S. degree in electronic information from the University of Science and Technology of China (USTC), Hefei, China, in 2024. His research interests include computational photography, depth sensing, and vision-language-action models for embodied intelligence.

Hanwei Zhu (Member, IEEE) received Ph.D. degree in computer science from the City University of Hong Kong, Hong Kong, in 2025. He is currently a research scientist with the Alibaba-NTU Global e-Sustainability CorpLab (ANGEL) at Nanyang Technological University. His research interests include perceptual image processing, computational vision, and computational photography.

Cong Zhang (Member, IEEE) received the Ph.D. degree from the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, the B.E. degree and the M.E. degree from the School of Artificial Intelligence, Optics and Electronics (iOPEN), Northwestern Polytechnical University. He was a Postdoctoral Fellow at The Hong Kong Polytechnic University and The Chinese University of Hong Kong. Currently, he is a Professor with the School of Control Science and Engineering, Shandong University.

Giuseppe Valenzise (Senior Member, IEEE) is a CNRS researcher at the Laboratoire des Signaux et Systèmes, CentraleSupélec, Université Paris-Saclay, France, where he leads the Multimedia and Networking team. He is Editor-in-Chief of the Springer EURASIP Journal on Image and Video Processing. He received his PhD from Politecnico di Milano, Italy. His research covers image and video processing, with a focus on compression (traditional and learning-based), light field and point cloud coding, quality assessment, high dynamic range imaging, and machine learning for visual analysis. He has co-authored over 100 publications in these areas and received the EURASIP Early Career Award in 2018. Giuseppe was General Chair of ICME 2025 and regularly serves on organizing committees of flagship conferences such as ICIP. He has been Associate Editor for IEEE TCSVT, IEEE TIP, and Signal Processing: Image Communication. He is currently Chair of the IEEE SPS Multimedia Signal Processing Technical Committee and a member of the IEEE CAS Multimedia Systems and Applications Technical Committee.



Weisi Lin (Fellow, IEEE) is a President's Chair Professor in Computer Science, Nanyang Technological University, Singapore. His research interests include intelligent image processing, perceptual signal modeling, video compression, and multimedia communication. He is a Chartered Engineer and a fellow of IET. He serves as a General Co-Chair for IEEE ICME 2025 and Lead General Chair for IEEE ICIP 2027, and was a Technical Program Chair of the IEEE ICME 2013, PCM 2012, QoMEX 2014/2026 and IEEE VCIP 2017. He has been a Keynote/Invited/Panelist/Tutorial Speaker at over 50 international conferences, and was a Distinguished Lecturer of the IEEE Circuits and Systems Society from 2016 to 2017 and the AsiaPacific Signal and Information Processing Association (APSIPA) from 2012 to 2013. He has been an Associate Editor of the IEEE Trans. Neural Networks and Learning Syst., the IEEE Trans. Image Process., the IEEE Trans. Circuits Syst. Video Technol., the IEEE Trans. Multimedia, and the IEEE Signal Process. Lett., as well as a Senior Editor for IEEE J. Selected Topics in Sig. Process. He has been awarded the Research Award 2023, College of Engineering, NTU, and is a Highly Cited Researcher since 2019 (awarded by Clarivate Analytics).

Kin-Man Lam (Senior Member, IEEE) received his Associateship in Electronic Engineering with distinction from Hong Kong Polytechnic in 1986, his M.Sc. degree in Communication Engineering from Imperial College in 1987, and his Ph.D. degree from the University of Sydney in 1996. He joined the Department of Electronic and Information Engineering, The Hong Kong Polytechnic University as an Assistant Professor in October 1996. He became an Associate Professor in 1999 and has been a Professor since 2010. He served as Chair of the IEEE Hong Kong Chapter of Signal Processing from 2006 to 2008. He was Technical Co-Chair of ISPACS 2005, PCM 2010, and IEEE VCIP 2020, and General Co-Chair of ICSPCC 2012, APSIPA ASC 2015, and IEEE ICME 2017. He also served as Director-Student Services (2012-2014), Director-Membership Services (2015-2017), and VP-Membership (2023-2025) for the IEEE SPS. Additionally, he held the roles of VP-Member Relations and Development (2014-2017) and VP-Publications (2017-2021) for APSIPA. He was an Associate Editor for IEEE Trans. on Image Processing (2009-2014) and Digital Signal Processing (2014-2018), an Editor for HKIE Transactions (2013-2018), and an Area Editor for the IEEE Signal Processing Magazine (2015-2017). He currently serves as a Senior Editorial Board Member of APSIPA Trans. on Signal and Information Processing and an Associate editor of the EURASIP International Journal on Image and Video Processing. His current research interests include image and video processing, computer vision, and human face analysis and recognition.